Pixels or Queues? A Careful Evaluation of Deep and Tabular Reinforcement Learning for Traffic Signal Control

Zahra Karami zahra.krm004@gmail.com
Azadeh Zamanifar azamanifar@iau.ac.ir
School of Electrical and Computer Engineering, Science and Research Branch,
Islamic Azad University, Tehran, Iran

Abstract

Adaptive traffic signal control is a natural testbed for reinforcement learning (RL), yet reported gains depend strongly on simulator details, reward design, and the metrics and baselines used for evaluation. We study two agents for an isolated four-approach intersection in a lightweight cellular simulator: a tabular Q-learning agent that observes queue counts, and a deep Q-network (DQN) that maps the raw $20\times20\times3$ occupancy raster directly to phase decisions. On 5,000 held-out episodes, the released DQN (a single training run) lowers the waiting-time measure used during development by 17.2% relative to tabular agents trained on the same grid for 3,000 episodes and by 44.9% relative to Webster-timed fixed-time control. We then broaden the evaluation with classical baselines, demand and asymmetry sweeps, per-vehicle delay tails, and a clearance-inclusive delay that also counts the all-red ticks inserted at every phase change. Between 31% and 65% of all halted vehicle-ticks fall in these clearance ticks, which the development measure never observes. Under full accounting, the comparison between the agents depends on how arrivals are simulated. When one arrival draw is reused for all ticks of a phase change, as in the released evaluation, the DQN is no better than tabular agents trained for 3,000 episodes and worse than tabular agents that also observe the current phase and are rewarded for clearance-inclusive delay, a change that cuts their switching by 71%. With independent arrivals in every tick, as in the DQN's own training environment, the DQN has 14.5% and 8.4% less delay than these two tabular agents. In both settings a longest-queue-first rule has the lowest clearance-inclusive delay, and the learned agents degrade quickly above their training demand. Saliency and input ablations show that the DQN relies on vehicles at the stop lines and on the signal-head pixels, from which it infers the current phase. Code, trained agents and arrival streams are released.

Keywords: traffic signal control; deep reinforcement learning; deep Q-network; Q-learning; reproducibility; evaluation methodology; interpretability.

1 Introduction

Traffic congestion imposes a large and persistent cost on urban travelers: in the United States the average commuter lost 54 hours to congestion delay in 2022, matching the pre-pandemic peak [35]. At signalized intersections, the allocation of green time between conflicting movements is one of the few levers that can be adjusted without new infrastructure. Classical practice relies on fixed-time plans derived from historical demand [20, 41], on actuated logic driven by detectors, and on adaptive systems such as SCOOT and SCATS [17, 33].

Reinforcement learning offers an alternative: a controller that learns a timing policy by interacting with traffic rather than from hand-crafted rules. Early work used model-based RL [45] and tabular or linear Q-learning with compact, engineered state descriptions [1, 9, 29]. The success of deep Q-networks on raw pixels [25] prompted a wave of deep RL controllers [10, 11, 21, 22, 26, 36, 42], and later methods that scale

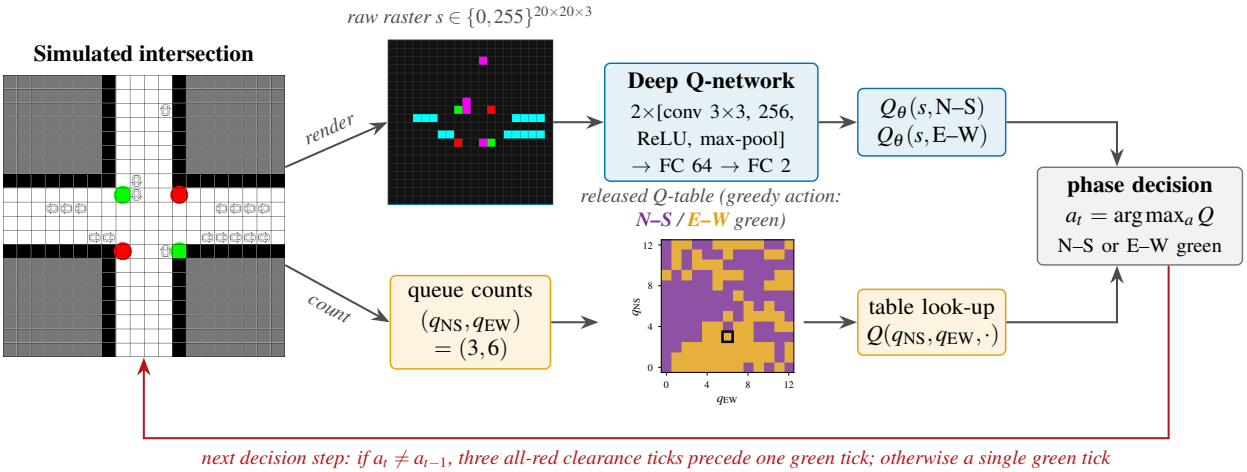


Figure 1: Two ways to see an intersection. The simulator (left, rendered with the original visualization code) is observed either as a raw 20×20×3 raster (scaled to $[0,1]$ before entering the network) that a convolutional deep Q-network maps to two action values (top), or as two queue counts that index a Q-table (bottom; the square marks the current state, colors show the greedy action of the released table). The chosen phase is applied at the next decision step; changing phase inserts three all-red clearance ticks.

to large networks [7, 8, 43, 49]; see [15, 28, 30, 44, 46] for surveys. At the same time, several studies have shown that conclusions about RL controllers are fragile: reward definitions that look reasonable need not reduce travel time [6, 50], rich state representations may be unnecessary [12, 50], and results change with sensing assumptions and baselines [3]. These observations echo broader concerns about evaluation in deep RL [2, 16].

This paper studies the question *"pixels or queues?"* in the most transparent setting we could find: a single four-approach intersection simulated by a cellular model in which every rule is visible in a few lines of code. We compare two agents developed for this environment (Figure 1). The first is a tabular Q-learning agent that observes only the number of vehicles approaching the stop lines on the north–south and east–west axes. The second is a convolutional DQN that receives the raw 20×20×3 raster of the intersection, including vehicles and signal heads, and must discover by itself which pixels matter. Starting from the released implementation, we first report the comparison under the protocol used during development, and then examine it with a deliberately broader evaluation.

Contributions.

- **Two agents in a transparent environment.** We formalize the cellular intersection, its reward and its metrics, and document a tabular and a pixel-based deep agent. Re-running the released evaluation reproduces the tabular agent's stored results exactly; the stored DQN results cannot be reproduced episode by episode, so we use our re-runs throughout (Section 6.1).
- **A broad, paired evaluation.** All controllers are evaluated on the same 5,000 held-out episodes against Webster-timed fixed-time control, self-organizing traffic lights (SOTL), a longest-queue-first (LQF) rule and a random policy, with confidence intervals, paired tests, seed-level comparisons and per-vehicle delay tails (Section 6.2).
- **Clearance-inclusive accounting.** The waiting-time measure used during development is evaluated only at decision ticks and misses the vehicles stopped during all-red clearance intervals. Counting every tick changes the conclusions and the ranking of controllers (Section 6.3).
- **What the agents learn.** Switching statistics, policy maps, gradient and occlusion saliency, and input

ablations show how the DQN responds to queue imbalance and that it reads stop-line occupancy and the signal state from pixels (Sections 6.4 and 6.5).

- **Robustness and design lessons.** Demand and asymmetry sweeps expose the limits of the learned policies (Section 6.6); a 2×2 ablation shows how a phase-aware state and a clearance-inclusive reward together make the tabular agent switch economically (Section 6.7); and a second arrival model shows which conclusions depend on how stochastic arrivals are implemented (Section 6.8).

2 Related work

Classical signal control. Fixed-time plans allocate green time from historical demand; Webster's formula gives the cycle length that minimizes delay at an isolated intersection under random arrivals [20, 41]. Adaptive systems such as SCOOT [17] and SCATS [33] adjust timings from detector data. Self-organizing traffic lights (SOTL) switch when the demand accumulated at a red light exceeds a threshold, subject to a minimum green [13]. Max-pressure control serves the phase with the largest difference between upstream and downstream queues and is throughput-optimal for networks of signalized intersections [38]; at an isolated intersection with free-flowing exits it serves the longer queue, which motivates the LQF rule we use as a strong, assumption-light baseline.

Reinforcement learning for signal control. Before deep RL, signal control was learned with model-based RL using car-based value functions [45] and with tabular or function-approximation Q-learning [1, 9, 29]. Deep agents have used discretized position and speed matrices [11, 22], image-like or aggregated traffic representations [26, 36], queue-length inputs with stacked autoencoders [21], experience replay and target networks [10], and real-world camera data [42]. Network-scale methods coordinate many intersections through pressure-based rewards [43], phase competition [49], parameter sharing [7] or multi-agent actor–critic training [8]. Most of these studies rely on microscopic simulators such as SUMO [24] or CityFlow [48]. Our environment is closer in spirit to cellular-automaton traffic models [5, 27]: deliberately simple, so that every effect we report can be traced to an explicit rule.

State, reward and evaluation. Genders and Razavi [12] found that low-resolution occupancy inputs perform almost as well as high-resolution vehicle positions and speeds, which bears directly on our question. Zheng et al. [50] showed that ad-hoc reward combinations need not minimize travel time and that simple states can suffice; Cabrejas-Egea et al. [6] compared reward functions under realistic sensing and found large differences in waiting time. Ault and Sharon [3] introduced the RESCO benchmark and observed that reported improvements were not robust to sensing assumptions or intersection layouts. In deep RL more broadly, Henderson et al. [16] and Agarwal et al. [2] documented the sensitivity of conclusions to seeds, implementation details and statistical reporting. We follow these recommendations by using multiple seeds, held-out arrival streams, paired comparisons and confidence intervals.

Interpreting learned controllers. Saliency methods based on gradients [32] or occlusion [47] have been adapted to visualize what deep RL agents attend to [14]. In traffic control, interpretability has also been pursued by restricting the policy class [4]. We use gradient and occlusion saliency together with targeted input ablations to test what the pixel-based agent actually relies on.

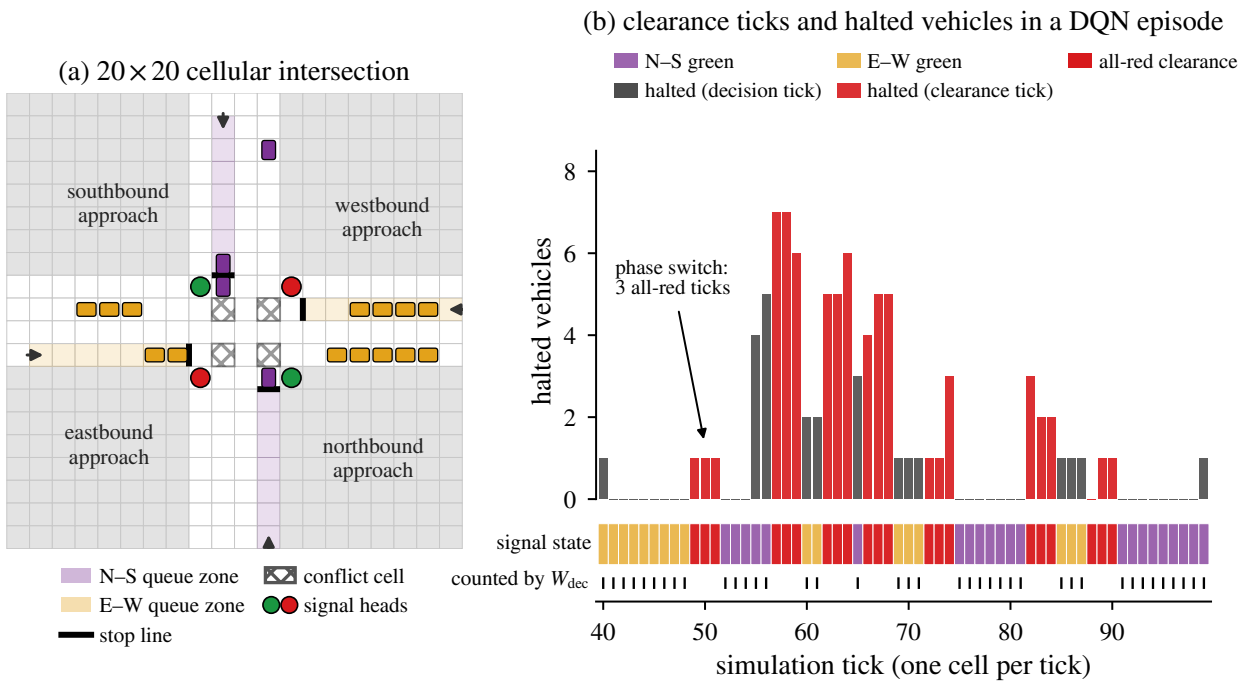


Figure 2: The cellular intersection. (a) Geometry of the default 20×20 grid with a snapshot of vehicles. Each approach is a single lane; vehicles enter at the boundary, queue behind the stop line while their signal is red, and leave at the opposite boundary. Shaded cells are counted in the queue of each approach; hatched cells are where crossing streams intersect. (b) A 60-tick excerpt of a DQN episode. Every phase change inserts three all-red ticks (red) before the new green tick; the development measure W_{dec} counts halted vehicles only at decision ticks (black marks), so the red bars are invisible to it.

3 Environment and evaluation measures

3.1 Geometry and vehicle dynamics

The intersection lives on an $S\times S$ cell grid with S=20 (a 15×15 variant is also used). Two roads cross at the center; each of the four approaches—northbound, southbound, eastbound and westbound—is a single lane without turning movements (Figure 2a). A vehicle normally occupies one cell (the exception is described in Section 3.2) and, at every simulation *tick*, advances by one cell unless (i) the cell ahead is occupied by a vehicle of the same approach or (ii) its signal is red and it is standing immediately before the signal line. Vehicles of an approach are updated in order of arrival, so a stopped platoon discharges at a saturation flow of one vehicle per tick without start-up loss. A vehicle that cannot move during a tick is counted as *halted*. Vehicles leave the grid at the far boundary. Crossing streams interact only through the signal: the all-red interval described below separates them.

3.2 Arrivals

At every decision step t, approach i receives a new vehicle at its entry cell if a uniform random number $u_{i,t}$ exceeds $1-p$ (default p=0.1) and the approach's queue is at most $S/2-2$ vehicles (8 on 20×20; 5 on 15×15, where $S/2-2=5.5$); otherwise the arrival is *blocked* and lost. An approach has only seven (respectively five) cells upstream of its signal line, so an arrival accepted at a full approach shares the entry cell with the vehicle already there; together with a queue-count detail of the released code (Appendix A), this allowed up to 10 (respectively 7) vehicles in one approach. Co-located vehicles occupy a single pixel of the

raster. The random numbers come from pre-generated streams, so that all controllers face identical draws (common random numbers). In the released evaluation, the draw $u_{i,t}$ of a decision step is applied to every tick of that step. The arrival rate per tick is therefore still p, but during a phase change the four ticks receive either no vehicle or one vehicle each, so switching steps create bursts of up to four arrivals per approach precisely while the intersection is blocked, and the tick at which a given draw takes effect depends on the controller's earlier switching. We keep this rule for our main results, to stay faithful to the released implementation, and repeat the key comparisons in Section 6.8 with an independent draw in every tick; there, tick k of an episode uses the k-th draw of the stream, so arrivals are identical in time for all controllers. The DQN's own training environment also draws arrivals independently in every tick.

3.3 Signal phases, decision steps and clearance

The controller chooses between two phases, $a \in \{0,1\}$: north–south green (east–west red) and east–west green (north–south red). If the chosen phase equals the current one, the decision step lasts one tick. If it differs, three all-red ticks are inserted to clear the intersection before one tick of the new phase, so the step lasts four ticks (Figure 2b). This makes switching costly: during clearance every vehicle at a stop line is halted. An episode consists of 200 decision steps, after which the grid is emptied.

3.4 Observations

Raster. The environment is rendered into a $20\times20\times3$ array: vehicles of the north–south and east–west approaches are written with two distinct color codes, and the four signal heads (placed at the corners of the intersection) with a green or red code (Figure 1). The DQN receives this array scaled to $[0,1]$. **Queues.** For approach i, the queue q_i counts the vehicles that are upstream of its signal line at the start of a tick, whether moving or halted. The tabular agent observes $q_{\mathrm{NS}} = q_{\mathrm{N}} + q_{\mathrm{S}}$ and $q_{\mathrm{EW}} = q_{\mathrm{E}} + q_{\mathrm{W}}$.

3.5 Rewards

Let h_t be the number of halted vehicles at the decision tick of step t (the last tick of the step) and q_i the queues after the step. The development reward, used for training the tabular agent and for evaluating both agents, is

$$r_t^{\mathrm{dev}} = \begin{cases} -2 & \text{if } h_t > h_{t-1}, \\ -1 & \text{if } h_t \le h_{t-1} \text{ and } \max_i q_i \ge S/4, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

The DQN was trained with a closely related queue-based reward: -2 unless the total queue $\sum_i q_i$ decreased during the step, and otherwise -1 if some queue strictly exceeds $S/4$ and 0 if none does. In Section 6.7 we also consider a clearance-inclusive delay reward $r_t^{\mathrm{delay}} = -\sum_{\tau\in t} h_\tau$, the total number of halted vehicle-ticks over *all* ticks τ of step t.

3.6 Evaluation measures

We report two waiting-time measures, both expressed in halted ticks per vehicle and computed per episode:

$$W_{\mathrm{dec}} = \frac{\sum_t h_t}{\sum_t n_t}, \qquad W_{\mathrm{all}} = \frac{\sum_\tau h_\tau}{\sum_\tau n_\tau}, \tag{2}$$

where t runs over decision ticks, τ over all ticks, and n counts arriving vehicles. W_{dec} is the measure computed by the released evaluation scripts; W_{all} adds the halted vehicles and the arrivals of the all-red clearance ticks.

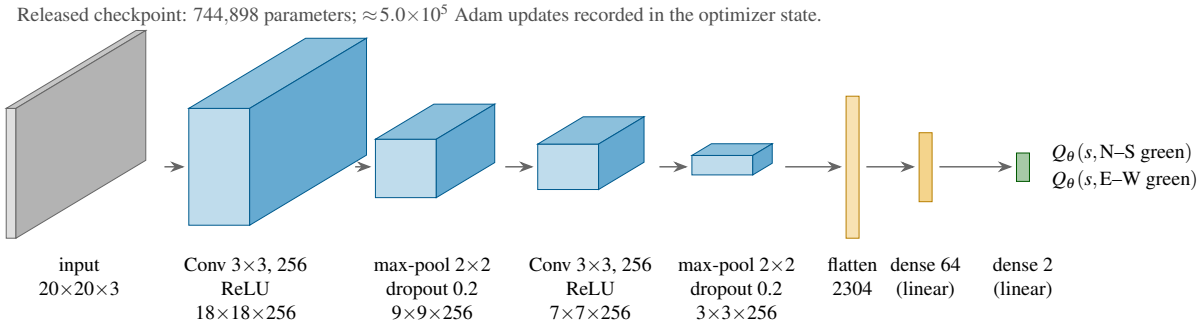


Figure 3: The pixel-based deep Q-network. Two convolution–ReLU–pooling blocks with 256 filters each are followed by a 64-unit linear layer and a two-unit output, one action value per phase. Shapes are for the 20×20 environment.

We also report the per-vehicle delay d_v (halted ticks of vehicle v between entry and exit), summarized by d_{95}, the 95th percentile within an episode averaged over episodes; throughput (vehicles leaving per tick); phase switches per episode; and the share of arrivals blocked at full approaches. Blocked arrivals accrue no delay, so at high demand W_{all} is favorable to controllers that block many arrivals.

4 Agents and baselines

4.1 Tabular Q-learning on queue counts

The tabular agent maintains $Q(q_{\text{NS}}, q_{\text{EW}}, a)$ and acts ε-greedily. After each step it applies the Q-learning update [40]

$$Q(s_t,a_t) \leftarrow (1-\alpha)\,Q(s_t,a_t) + \alpha\left(r_t^{\text{dev}} + \gamma \max_{a'} Q(s_{t+1},a')\right), \tag{3}$$

with α=0.1, γ=0.95 and the bootstrap term omitted at the end of an episode. The released training script runs 100 episodes of 200 steps on the 15×15 grid with ε starting at 1 and decaying by a factor of 0.995 per episode, and initializes unseen entries to zero. The released table, however, contains all 40×40 queue pairs, and its entries outside the visited region hold values in $[-3,0]$ consistent with a random initialization; together with Section 6.7, this indicates that it was trained with different settings from those in the released script. Its greedy policy is shown in Figure 1.

4.2 Deep Q-network on raw pixels

The DQN [25] maps the raster to two action values with two convolutional blocks and two dense layers (Figure 3). The released training script uses experience replay [23] (capacity 50,000 transitions, warm-up 1,000, minibatches of 64), a target network synchronized every three episodes, γ=0.95, a mean-squared temporal-difference loss and Adam [19] with learning rate 10^{-3}; dropout [34] is applied after each pooling layer during training only. Training episodes last 1,000 steps, arrivals are drawn independently in every tick with p=0.1, and ε starts at 0.9 and decays by a factor of 0.9998 per episode. The optimizer state of the released checkpoint records 499,701 parameter updates, i.e., roughly 500 training episodes; under the released schedule ε would still be about 0.81 after 500 episodes, so the network would have been learned largely from exploratory behavior, which off-policy Q-learning permits. The schedule actually used for the checkpoint is not recorded.

4.3 Classical baselines

All baseline parameters are tuned to minimize W_{all} on a separate tuning stream (Section 5). **Fixed-time (Webster).** Phases alternate after g decision steps each, so a cycle lasts $2(g+3)$ ticks including six ticks of clearance. Webster's optimal cycle is $C_0 = (1.5L+5)/(1-Y)$ seconds [41]; taking one tick as one second, the lost time is L=6 and, with a saturation flow of one vehicle per tick, the sum of critical flow ratios is $Y = p_{\text{NS}} + p_{\text{EW}} = 0.2$. This gives $C_0 = 17.5$ ticks and $g \approx 5.75$. A grid search over $g \in \{1,\dots,20\}$ selects g=6 on both grids, in agreement with the formula. For demand sweeps we additionally re-time the plan with Webster's formula at each demand level ("Webster, re-timed"). **SOTL.** We use a variant of the self-organizing rule of Gershenson [13] that counts queued vehicles rather than accumulating vehicle–time: the light switches when at least θ vehicles queue on the red approaches and the current green has lasted at least $g_{\min}$ steps; tuning gives $(\theta, g_{\min}) = (3,5)$ on 20×20 and $(2,4)$ on 15×15. **Longest-queue-first (LQF).** The axis with the larger queue receives green, ties keep the current phase, and a minimum green of $g_{\min}$ steps is enforced ($g_{\min}$=3 on 20×20, 2 on 15×15). LQF therefore uses the two queue counts, the current phase and a green timer. Because downstream links never queue in this environment, max-pressure control [38] would select the same phase as LQF without the minimum green and the tie rule. **Random** chooses each phase with probability $1/2$.

5 Experimental protocol

Released protocol. The repository evaluates each released agent on its own grid (15×15 for Q-learning, 20×20 for the DQN) over 1,000 consecutive episodes of a single pre-generated arrival stream. We reproduce this protocol exactly, including the initial signal state of the first episode. Note that the tabular agent was trained on the first 20,000 steps of the same stream, so the first 100 test episodes overlap with its training data.

Held-out protocol. All other results use five independent test streams (seeds 2001–2005), each with 1,000 episodes of 200 decision steps (5,000 episodes in total). Every controller faces exactly the same random draws, which enables paired comparisons. Baseline parameters are tuned on a separate stream (seed 1001, 300 episodes), and learning curves are evaluated on a third stream (seed 3001, 50 episodes). For single controllers we report 95% bootstrap confidence intervals (4,000 resamples) for means and paired differences, together with paired t-tests over episodes.

Retraining the tabular agent. We retrain Q-learning with ten seeds per configuration, each with its own training stream: the released settings (100 episodes) and longer training (3,000 episodes, ε floored at 0.01) on both grids, plus three design variants on 20×20 that add the current phase to the state, replace r^{dev} by r^{delay}, or both. We denote tabular agents by Q: Q (released), Q (100 ep.), Q (3000 ep.), Q + phase, Q + delay and Q + phase + delay, where the last three are trained for 3,000 episodes. For retrained agents we report the mean and standard deviation across seeds, and comparisons with the DQN are made at the level of seeds (a t-interval over the ten seed means), because training-seed variability exceeds the episode-level uncertainty. The DQN is a single released checkpoint; retraining it was beyond our compute budget (Section 7), so its seed-to-seed variability is unknown.

Stress tests. Demand sweeps vary $p \in \{0.02, 0.04, \dots, 0.20, 0.25, 0.30\}$ on all approaches (two streams of 250 episodes per level); asymmetric sweeps vary $p_{\text{NS}}, p_{\text{EW}} \in \{0.04, 0.07, 0.10, 0.13, 0.16\}$ independently (250 episodes per cell). Learned agents are applied without retraining; retrained tabular agents are represented by all ten seeds.

Table 1: Released protocol. Mean episode reward and W_{dec} over the 1,000 released test episodes, as stored in the repository and when re-running the released code and models.

	Q-learning (15×15)		DQN (20×20)	
	Reward	W_{dec}	Reward	W_{dec}
Stored results in the repository	−108.466	3.110	−92.845	2.703
Re-run of released code and models	−108.466	3.110	−93.551	2.786
Episodes with identical reward	1000/1000		13/1000	
Re-run, clearance-inclusive W_{all}	5.133		5.138	
Re-run, phase switches per episode	47.1		33.8	

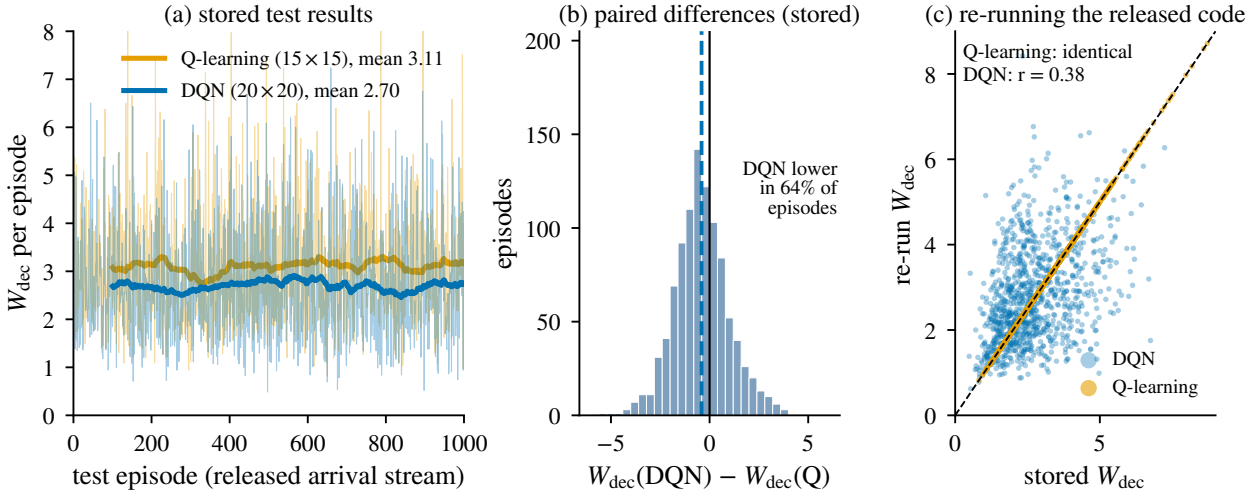


Figure 4: Released results. (a) Stored per-episode W_{dec} of both agents with 100-episode moving averages. (b) Paired differences on matched episodes. (c) Stored versus re-run W_{dec}: the tabular agent is reproduced exactly; the DQN re-run has a similar mean but only moderately correlated episodes.

Second arrival model. To test the dependence on the reuse of arrival draws, we repeat the main comparison with independent draws in every tick (test stream seed 6001, 1,000 episodes). Baselines are re-tuned under this model on a separate stream (fixed-time g=5, LQF $g_{\min}$=3, SOTL $(2,4)$), and the tabular agents are retrained under it with ten new seeds.

Implementation and validation. For speed and instrumentation we re-implemented the simulator, keeping its update order and arrival rule. Re-running the released Q-table through it reproduces all 1,000 stored episode rewards and waiting times exactly. The DQN forward pass was re-implemented in NumPy and validated against Keras on 3,000 environment states (maximum absolute difference 4×10^{-6}; identical greedy actions). All experiments together take about two hours on a four-core CPU.

6 Results

6.1 Reproducing the released evaluation

Table 1 and Figure 4 summarize the comparison as it was originally carried out. The stored results show the DQN with a mean W_{dec} of 2.70 against 3.11 for Q-learning, a 13.1% reduction, with the DQN better in 64% of matched episodes (paired t-test, $p = 1.6 \times 10^{-19}$). Re-running the released evaluation of the Q-table

Table 2: Main results on the 20×20 grid (5,000 held-out episodes; mean ± 95% CI half-width for single controllers, mean ± s.d. across 10 seeds for retrained agents). d_{95}: per-episode 95th percentile of vehicle delay, averaged over episodes. †Released table applied without retraining to the larger grid.

Controller	W_{dec} (development)	W_{all} (all ticks)	d_{95} (ticks)	Throughput (veh./tick)	Switches (per ep.)	Blocked (%)
Classical controllers (tuned on W_{all})						
Fixed-time (Webster, g=6)	5.00 ± 0.02	5.89 ± 0.02	12.9	0.365	33.0	0.3
SOTL	3.72 ± 0.02	4.79 ± 0.02	14.3	0.364	19.5	0.2
LQF	2.46 ± 0.02	3.63 ± 0.02	11.9	0.369	27.4	0.1
Random	10.42 ± 0.06	15.06 ± 0.06	38.0	0.359	99.8	3.7
Released agents						
DQN (pixels)	2.76 ± 0.03	5.08 ± 0.05	16.5	0.366	33.7	0.6
Q (released, transferred)†	3.80 ± 0.05	6.19 ± 0.07	20.4	0.367	46.0	0.9
Tabular Q-learning retrained on 20×20 (mean ± s.d. across 10 seeds)						
Q (100 ep.)	3.68 ± 0.24	5.93 ± 0.43	18.7	0.368	45.8	0.7
Q (3000 ep.)	3.33 ± 0.10	5.12 ± 0.17	16.7	0.369	42.5	0.4
Q + phase + delay	3.93 ± 0.30	4.73 ± 0.31	17.8	0.361	12.5	0.2

reproduces every stored episode exactly. Re-running the released DQN checkpoint gives a similar mean ($W_{\text{dec}} = 2.79$ instead of 2.70), but the stored results cannot be reproduced episode by episode: only 13 of 1,000 episode rewards coincide, about as many as between the DQN and the Q-table themselves (10–11), and the per-episode correlation between stored and re-run values is 0.38, comparable to that between two different controllers on the same arrivals (0.27–0.30). Two plausible explanations do not account for the difference: evaluating with active dropout or with ε=0.05 exploration gives mean W_{dec} values of 2.84 and 3.20 on the first 200 episodes (deterministic re-run 2.83, stored 2.68). We therefore use our re-runs, which apply a single pipeline to every controller, throughout; with re-run values the reduction is 10.4%. The last two rows of Table 1 preview a main theme of this paper: once clearance ticks are counted, the two agents are almost indistinguishable (5.14 versus 5.13 halted ticks per vehicle).

6.2 Main comparison on held-out traffic

Table 2 and Figure 5 report all controllers on the 20×20 grid, and Table 3 the comparisons with the DQN. Under the development measure, the released DQN checkpoint outperforms the other learned agents and fixed-time control on fresh traffic. On the common 20×20 grid it reduces W_{dec} by 17.2% relative to tabular agents trained for 3,000 episodes and by 25.2% relative to agents trained with the released script; it is better than all ten seeds of both. Relative to the Webster-timed fixed-time plan the reduction is 44.9%. In the configuration of the original comparison—each released agent on its own grid—the reduction relative to the tabular agent is 11.7%. The released Q-table transfers poorly to the larger grid ($W_{\text{dec}} = 3.80$), because queue counts there exceed the range it visited during training.

Two observations qualify this result. First, the LQF rule, which uses only the two queue counts, the current phase and a green timer, is better than every learned controller on both measures: the DQN's W_{dec} is 12.1% higher than LQF's. Second, the gaps change markedly when clearance ticks are counted. Under W_{all} the DQN's advantage over the long-trained tabular agents vanishes (relative difference −0.7%, $p = 0.50$ over seeds; better than 6 of 10 seeds), its advantage over fixed-time control falls to 13.8%, SOTL moves ahead of it, and the DQN's W_{all} is 39.8% higher than LQF's. Tuning SOTL on W_{dec} instead of W_{all} would lower its W_{dec} by a further 5%, which does not change these conclusions. The delay distribution (Figure 5b) adds a fairness perspective: the learned agents produce long tails, with 3.6% of vehicles under the DQN waiting more than 20 ticks, compared with 0.6% under LQF and 0.7% under fixed-time control, whose regular cycle

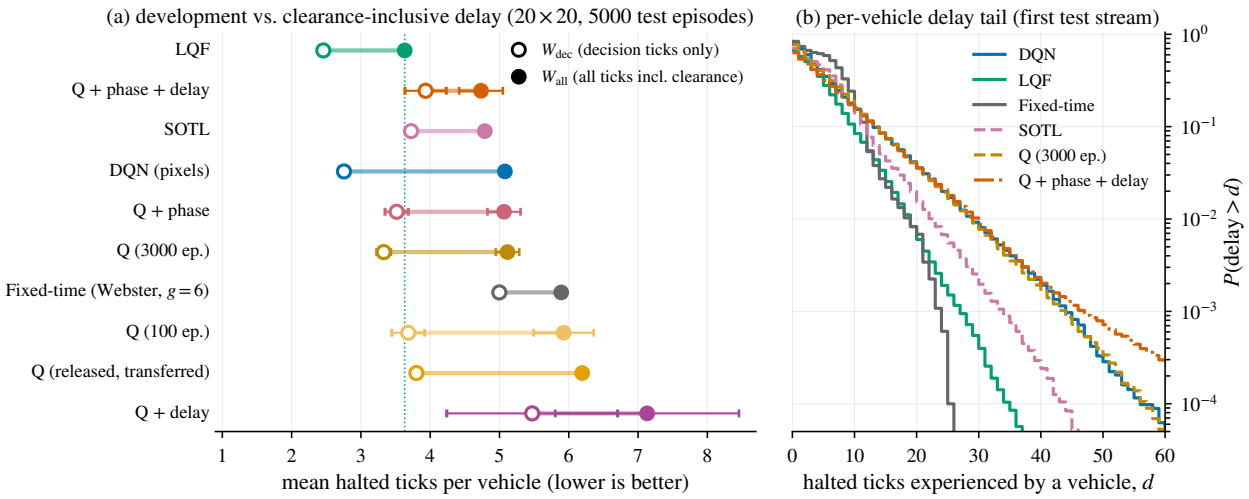


Figure 5: Held-out evaluation on the 20×20 **grid.** (a) For each controller, the open marker is the development measure W_{dec} and the filled marker the clearance-inclusive W_{all} (error bars: 95% CI, or across-seed s.d. for retrained agents); the random policy ($W_{\text{dec}} = 10.42$, $W_{\text{all}} = 15.06$) is omitted for scale. The dotted line marks LQF's W_{all}. (b) Survival function of per-vehicle delay on the first test stream (1,000 episodes; 200 per seed for retrained agents, pooled over ten seeds).

Table 3: Comparisons with the DQN on the 5,000 held-out episodes (20×20). Δ is the mean difference DQN minus controller (negative favors the DQN). For single controllers, CIs are paired bootstrap intervals over episodes, p-values come from paired t-tests, and "DQN better" is the share of episodes in which the DQN has the lower value. For retrained agents, the comparison is with the ten seed means (t-interval and one-sample t-test). ‡Matched episodes on different grids, as in the original comparison.

	W_{dec} (development measure)			W_{all} (clearance-inclusive)		
DQN vs.	Δ [95% CI]	p	DQN better	Δ [95% CI]	p	DQN better
Single controllers: paired over 5,000 episodes (bootstrap CI, paired t-test, share of episodes)						
LQF	+0.30 [+0.27, +0.33]	7.5×10^{-74}	42%	+1.45 [+1.40, +1.49]	$< 10^{-300}$	17%
SOTL	−0.97 [−1.00, −0.93]	$< 10^{-300}$	80%	+0.29 [+0.24, +0.33]	1.4×10^{-34}	49%
Fixed-time (Webster)	−2.24 [−2.28, −2.21]	$< 10^{-300}$	95%	−0.81 [−0.86, −0.76]	1.8×10^{-225}	72%
Q (released, transferred)	−1.04 [−1.09, −1.00]	$< 10^{-300}$	74%	−1.12 [−1.19, −1.05]	3.0×10^{-196}	67%
Q (released) on its 15×15 grid‡	−0.37 [−0.40, −0.33]	9.0×10^{-74}	61%	−0.07 [−0.12, −0.01]	0.020	53%
Retrained tabular agents: over 10 training seeds (t-interval, one-sample t-test, share of seeds)						
Q (100 ep.)	−0.93 [−1.10, −0.76]	6.1×10^{-7}	10/10	−0.85 [−1.16, −0.54]	1.6×10^{-4}	10/10
Q (3000 ep.)	−0.57 [−0.64, −0.50]	2.1×10^{-8}	10/10	−0.04 [−0.16, +0.08]	0.503	6/10
Q + phase	−0.76 [−0.88, −0.64]	1.6×10^{-7}	10/10	+0.01 [−0.16, +0.18]	0.870	4/10
Q + phase + delay	−1.18 [−1.39, −0.97]	5.3×10^{-7}	10/10	+0.35 [+0.12, +0.57]	0.007	1/10

caps the worst case.

6.3 Where the delay accrues: clearance-inclusive accounting

Why do the two measures disagree? Figure 6a decomposes the delay of each controller. Between 31% (Q + phase + delay) and 65% (DQN) of all halted vehicle-ticks occur during all-red clearance, i.e., in ticks that W_{dec} never observes. The share is large for the learned agents because of *how* they switch, not only how often: the DQN switches 33.7 times per episode, 23% more often than LQF (27.4), and halts 4.0 vehicles per clearance tick against LQF's 2.7. Its green-phase delay (1.84 ticks per vehicle) is close to LQF's (1.76), so most of the gap between the two lies in clearance (3.46 versus 1.98).

Figure 6b shows the consequence for evaluation. Across 92 parameterized baselines the two measures

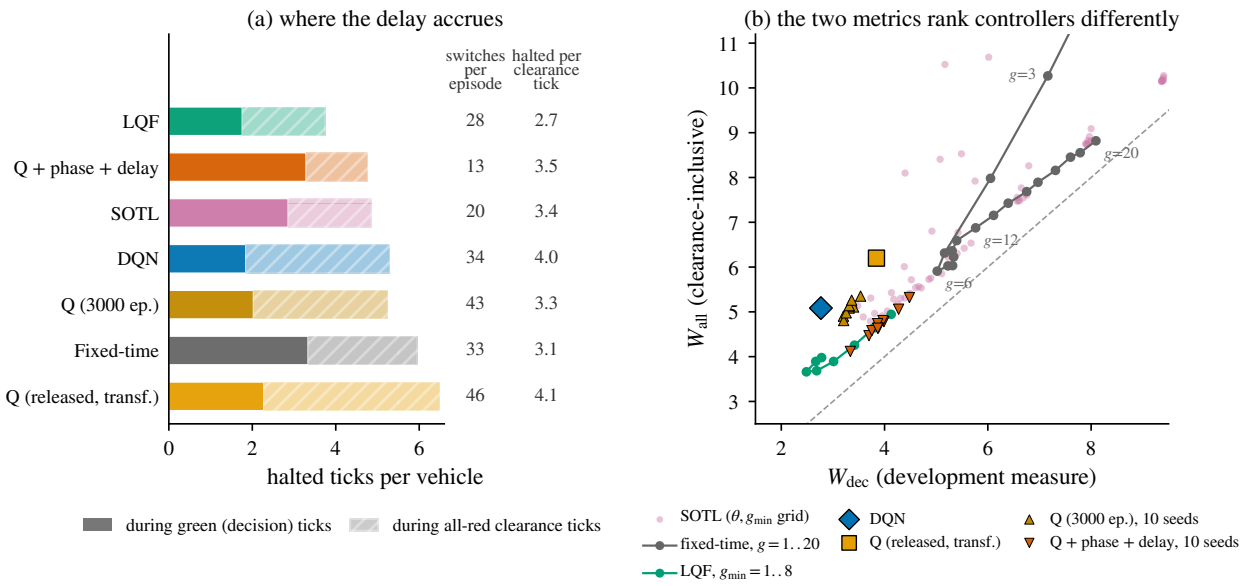


Figure 6: Accounting for clearance. (a) Per-vehicle delay split into ticks halted under a green phase and ticks halted during all-red clearance, pooled over the first test stream (1,000 episodes; for retrained agents 200 per seed over ten seeds). Because these are pooled ratios they differ slightly from the per-episode means of Table 2. Columns give phase switches per episode and the mean number of vehicles halted per clearance tick. (b) W_{dec} versus W_{all} for fixed-time plans ($g = 1, \ldots, 20$), LQF with $g_{min} = 1, \ldots, 8$, the 8×8 SOTL grid and the learned agents. Points far above the dashed diagonal are controllers whose delay W_{dec} understates most; because W_{dec} also omits the arrivals of clearance ticks, the gap is smaller than the clearance delay itself.

are correlated, but the offset between them depends on the switching behavior, so rankings change: the DQN has the second-lowest W_{dec} among the controllers of Table 2, but under W_{all} it is beaten by LQF, SOTL and Q + phase + delay and statistically indistinguishable from Q (3000 ep.) and Q + phase (Table 3). A reward or metric that ignores clearance rewards precisely the behavior that makes a controller expensive.

6.4 What the agents learn

Figure 7 characterizes how each controller switches. The tabular agents without phase information follow the queue imbalance closely: when the opposing queue is one or two vehicles longer, the 3,000-episode agents switch in 88% of decisions and the released table, on its 15×15 grid, in 97%, close to the pure longest-queue rule, with which 90% of the released table's decisions coincide (Figure 8). The DQN responds much more gradually. For the same imbalance it switches in only 30% of decisions (LQF: 84%), yet when the green axis holds strictly more vehicles than the red one it still switches in 7.5% of decisions, against 1.8% for the 3,000-episode tabular agents and never for LQF. These switches away from the longer queue produce many short phases (median 3 steps against 5 for LQF; mean 5.6 against 7.0); the tabular agents' phases are even shorter, mainly because they often switch when the two queues are equal (28% of tied decisions). At the same time, the DQN's reluctance to switch for moderate imbalances lets queues build up, so that its switches occur with more vehicles waiting (5.5 against 4.0 for LQF). Each switch therefore stops more vehicles for three ticks, which explains much of the excess clearance delay in Figure 6a. The DQN's map in Figure 8 shows the same pattern: decisions vary within cells, because it conditions on vehicle positions rather than counts alone, and the band in which it keeps the current phase against a longer opposing queue is slightly wider while east–west has green. Q + phase + delay sits at the other extreme: it rarely switches unless the opposing queue is much longer, so its phases last 14.9 steps on average, which keeps clearance delay low but

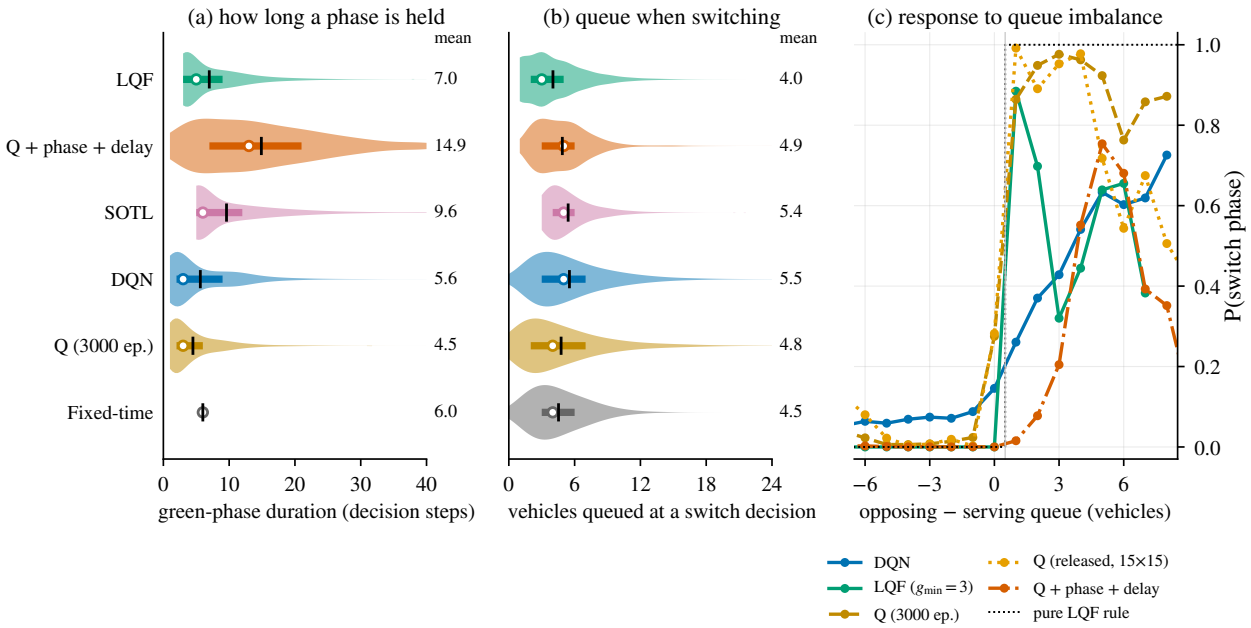


Figure 7: Switching behavior (20×20, first test stream). (a) Distribution of green-phase durations in decision steps. (b) Total number of queued vehicles at decisions that trigger a switch. Bars: interquartile range; white dot: median; vertical tick: mean. (c) Probability of switching as a function of the queue imbalance between the red and the green axis. LQF's non-monotone curve reflects its minimum green: large imbalances arise mostly just after a switch, when switching is not yet allowed. Retrained agents are pooled over ten seeds; the released Q-table is shown on its own 15×15 grid.

lets queues grow during green.

6.5 What the DQN looks at

Because the DQN is never told what a queue or a phase is, we probed which parts of its input drive its decisions (Figure 9). Gradient saliency [32] concentrates on the intersection box and the cells around the stop lines (panel a). Occlusion [14, 47], which removes one object at a time from 600 visited states, makes this precise (panels b and d): a north–south vehicle standing at its stop line shifts $Q_{\mathrm{NS}} - Q_{\mathrm{EW}}$ by $+0.54$ on average and an east–west vehicle by -0.46, the vehicles one cell behind by $+0.23$ and -0.21, and vehicles further upstream or past the signal have small effects, except for vehicles in the cells next to the grid boundary (-0.20 and $+0.32$), probably an edge effect of the unpadded convolutions. The network has thus learned a short-sighted but sensible feature: *is someone waiting at the line, and on which axis?*

The signal-head pixels carry the second ingredient (panel e). While east–west has green, the red north–south heads and the green east–west heads push the decision towards east–west (-0.30 and -0.31, together -0.61); while north–south has green, the heads push towards north–south, but much more weakly (together $+0.18$). The lights therefore encode a preference for the current phase that is strong during east–west green and weak during north–south green, consistent with the asymmetric bands in Figure 8. Corrupting these pixels at test time confirms that the agent depends on them (panel c): removing the heads raises W_{all} from 5.19 to 7.93 and switching from 34 to 57 per episode, and swapping their colors raises them to 9.77 and 105. The DQN has learned to read the signal state from pixels—information that the original tabular state lacks.

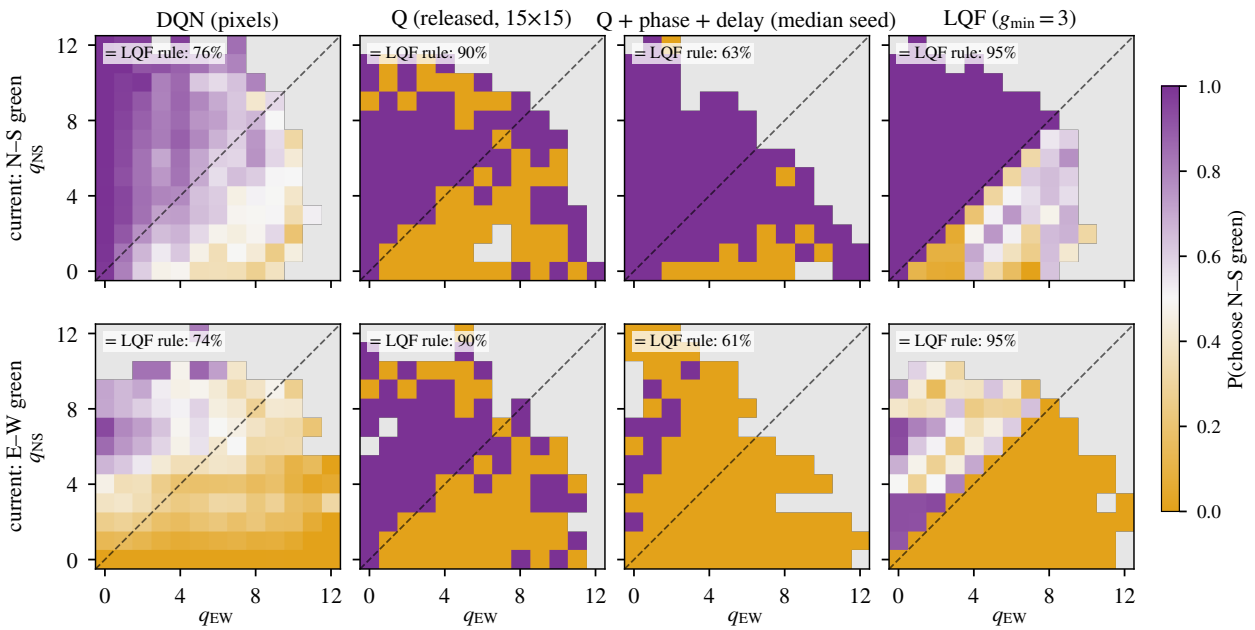


Figure 8: Policy maps. Probability of choosing north–south green as a function of the queue counts, separately for the two current phases (rows), estimated from 200,000 decisions per controller (cells with fewer than 15 visits in gray). The dashed diagonal is the LQF switching boundary; the inset gives the share of decisions that coincide with the pure longest-queue rule (no minimum green). The released Q-table has no phase information and therefore the same map in both rows; for Q + phase + delay we show the seed with the median W_{all}.

6.6 Robustness to demand

The agents were trained at a single demand level. Figure 10 applies them unchanged to lighter and heavier traffic. At light demand the DQN is much better than fixed-time control: at $p = 0.02$ its W_{all} is 1.66, against 4.74 for fixed-time control (LQF: 0.70). Above the training demand, however, all learned agents deteriorate quickly. At $p = 0.12$ the DQN and the fixed-time plan tuned at $p = 0.1$ are nearly equal (6.38 and 6.31); at $p = 0.14$ the DQN's W_{all} is 8.02 against 6.78 (fixed-time) and 5.05 (LQF), and at $p = 0.30$ it reaches 22.29 while the re-timed Webster plan achieves 11.59. Because blocked arrivals accrue no delay, these values understate the DQN's disadvantage: at $p = 0.30$, 30% of arrivals are blocked under the DQN against 10% under the re-timed plan. LQF, whose minimum green we did not re-tune, remains the best controller up to $p = 0.20$ and is then overtaken by fixed-time plans, whose long cycles waste less time in clearance; at $p = 0.30$ even the plan tuned at $p = 0.1$ is better (13.15 against 14.17). Webster's formula is only an approximation in this discrete setting: at light demand its short cycles are slightly worse than the plan tuned at $p = 0.1$. Q + phase + delay, which switches rarely, degrades more gracefully than the other learned agents but is poor at very light demand (9.30 at $p = 0.02$), where its long phases leave vehicles waiting at an otherwise empty intersection.

Under asymmetric demand (Figure 11) LQF is the best of the eight controllers in all 25 demand combinations. The DQN adapts to imbalance—it beats the equal-split fixed-time plan in 20 of 25 cells and the demand-aware Webster plan in 19—and it outperforms Q (3000 ep.) in 22 cells, while Q + phase + delay beats it in 12 cells, mostly at higher total demand. As in the symmetric sweep, the DQN's delay grows steeply with the total demand.

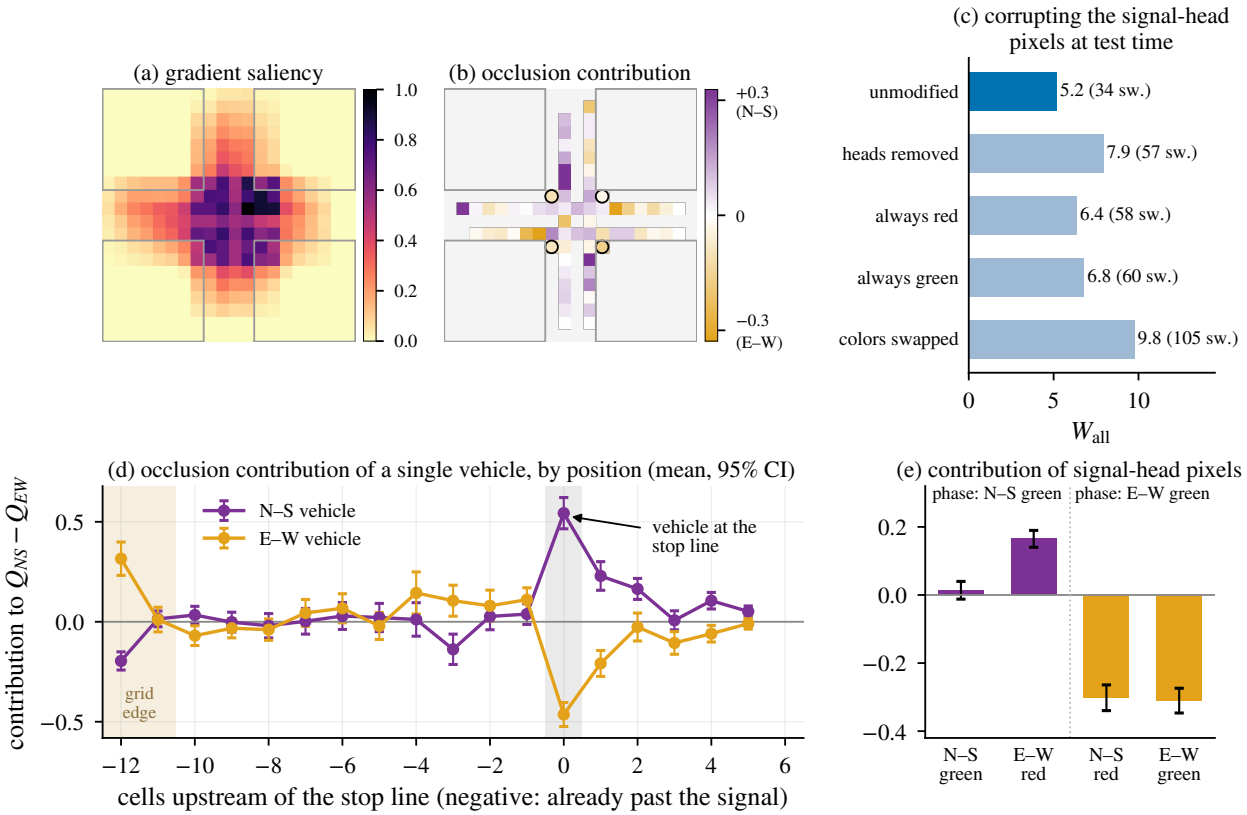


Figure 9: Interpreting the pixel-based agent. (a) Mean absolute gradient of $Q_{\mathrm{NS}} - Q_{\mathrm{EW}}$ with respect to the input, over about 5,000 visited states (normalized). (b) Mean occlusion contribution of an object in each cell, i.e., the change in $Q_{\mathrm{NS}} - Q_{\mathrm{EW}}$ caused by its presence (purple: favors north–south green; orange: east–west); circles mark the signal heads. (c) Clearance-inclusive delay and switches when the signal-head pixels are corrupted at test time (250 episodes). (d) Contribution of a single vehicle as a function of its distance to the stop line; the shaded band at the left marks cells adjacent to the grid boundary. (e) Contribution of the signal-head pixels in each phase.

Table 4: State and reward design for tabular Q-learning (20×20, 3,000 training episodes, mean ± s.d. across ten seeds, 5,000 held-out episodes).

Agent	State	Reward	W_{dec}	W_{all}	Switches/ep.	d_{95}
Q (3000 ep.)	queues	r^{dev}	3.33 ± 0.10	5.12 ± 0.17	42.5	16.7
Q + phase	queues + phase	r^{dev}	3.52 ± 0.17	5.07 ± 0.24	23.5	18.5
Q + delay	queues	r^{delay}	5.47 ± 1.23	7.13 ± 1.33	33.5	23.9
Q + phase + delay	queues + phase	r^{delay}	3.93 ± 0.30	4.73 ± 0.31	12.5	17.8

6.7 Training dynamics and design of the tabular agent

Figure 12 shows that the released training budget of 100 episodes is far from convergence: agents trained for 3,000 episodes reach $W_{\mathrm{all}} = 5.12$ instead of 5.93 (Table 2). On the 15×15 grid, long training matches the released Q-table (Appendix C), which again suggests that the latter was trained for longer than the released script specifies.

We then asked why the tabular agent does not learn to switch economically. Table 4 crosses two design choices. Adding the current phase to the state reduces switching by 45% but leaves W_{all} essentially unchanged, and replacing the reward alone makes learning unstable (Figure 12b). Combining both reduces W_{all} by 7.5% to 4.73 and cuts switching by 71%, from 42.5 to 12.5 per episode; this is below the released DQN, whose

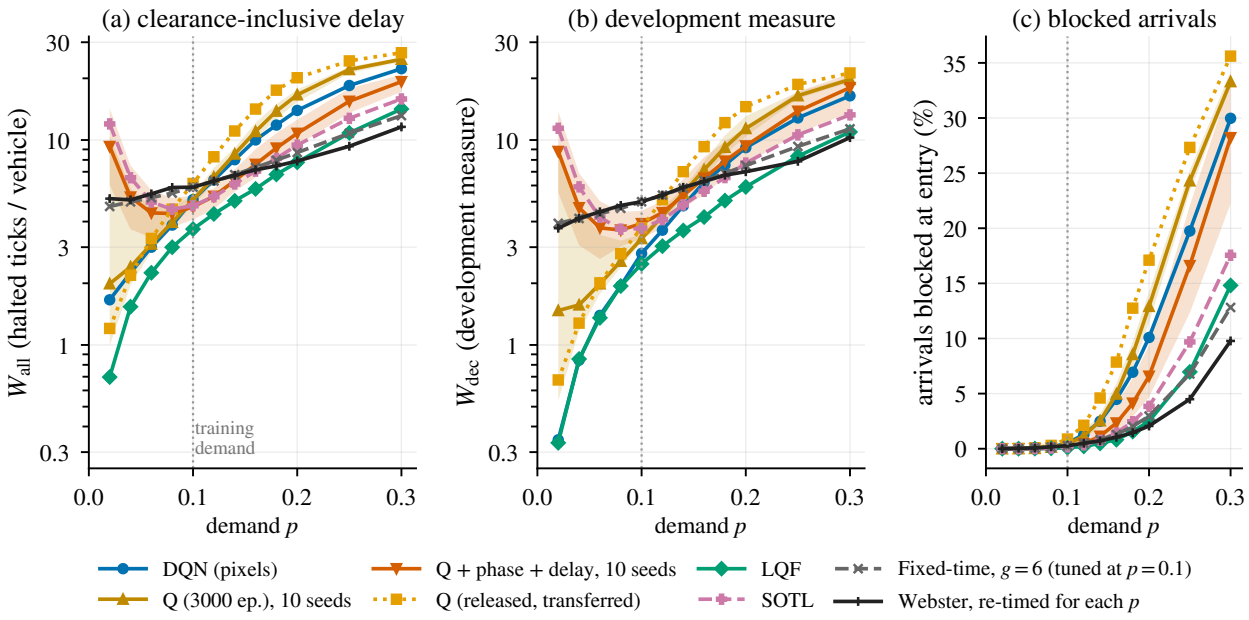


Figure 10: Demand sweep (20×20; agents trained at $p = 0.1$, dotted line; p is the arrival probability per approach and decision step). (a) Clearance-inclusive delay and (b) development measure on log scales; (c) share of arrivals blocked at full approaches. Shaded bands span the ten training seeds of the retrained tabular agents.

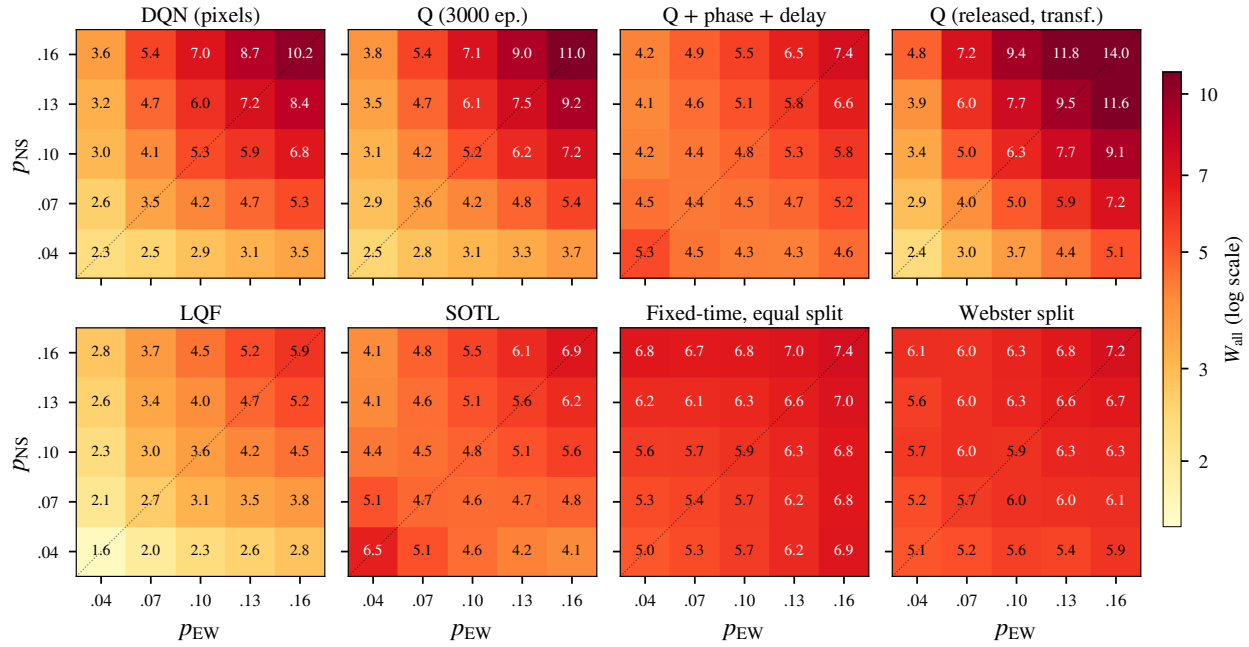


Figure 11: Asymmetric demand (20×20, 250 episodes per cell). Clearance-inclusive delay W_{all} for independent arrival probabilities on the north–south (p_{NS}) and east–west (p_{EW}) approaches. The fixed-time plan uses equal splits; the Webster plan allocates green in proportion to demand. Retrained tabular agents are averaged over ten seeds.

W_{all} is 7.3% higher, although the DQN remains clearly better under W_{dec}. The explanation is simple: without the phase, the agent cannot tell whether an action will trigger a clearance interval; with the development reward, the cost of that interval enters only indirectly, through the halted count at the following decision tick. Only with both does less switching translate into lower delay. A gap to LQF (3.63) remains, and it is

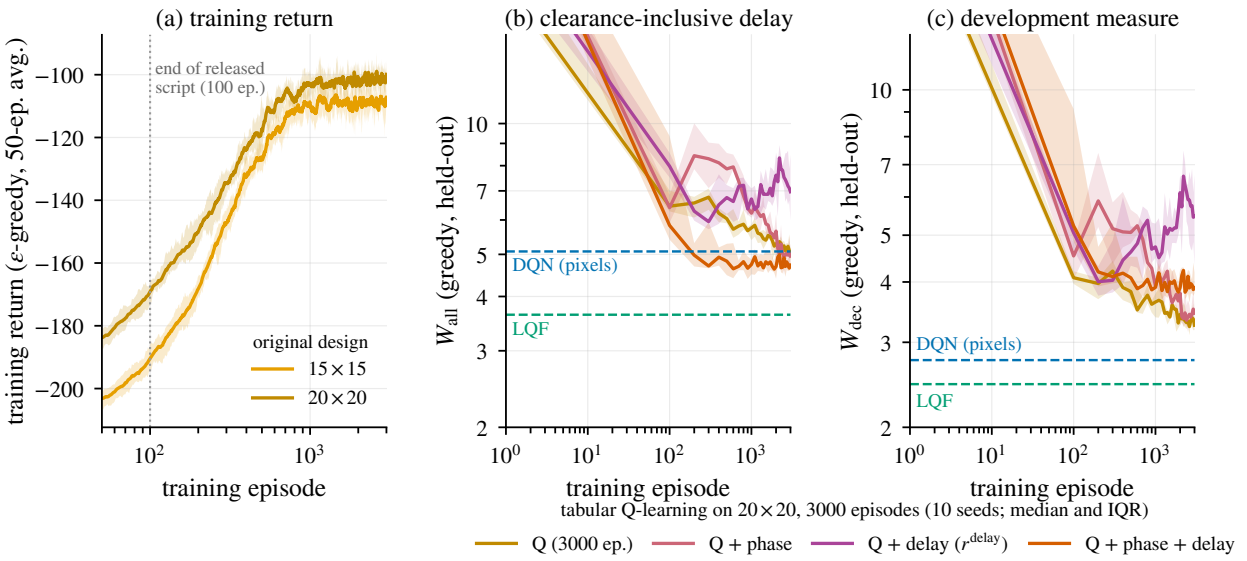


Figure 12: Learning curves of tabular Q-learning. (a) Training return of the original design on both grids (ten seeds; band: 10th–90th percentile). The return is measured under the ε-greedy behavior policy, so part of its rise reflects the decaying ε, which reaches its floor after about 920 episodes. (b, c) Greedy performance on a held-out stream for four designs on 20×20 (median and interquartile range over ten seeds); dashed lines show the released DQN and LQF.

Table 5: Two arrival models (20×20). Left: released model, in which a decision step's arrival draw is reused for every tick of a phase change (values from Table 2). Right: independent draws in every tick (1,000 held-out episodes; baselines re-tuned). The last column gives the DQN's relative difference in W_{all} (negative: DQN better). *Retrained under each arrival model (ten seeds each; means shown).

	Released draws (per decision step)		Independent draws (per tick)			
Controller	W_{dec}	W_{all}	W_{dec}	W_{all}	Switches	DQN vs. (W_{all})
LQF	2.46	3.63	2.27 ± 0.03	3.21 ± 0.03	32.8	+16.7%
SOTL	3.72	4.79	2.89 ± 0.03	4.06 ± 0.03	35.0	−7.7%
Fixed-time	5.00	5.89	3.60 ± 0.03	4.62 ± 0.03	39.0	−19.0%
DQN (pixels)	2.76	5.08	2.11 ± 0.03	3.74 ± 0.04	41.7	—
Q (released, transferred)	3.80	6.19	2.87 ± 0.03	4.36 ± 0.04	54.5	−14.1%
Q (3000 ep.)*	3.33	5.12	2.93 ± 0.12	4.38 ± 0.14	54.1	−14.5% (10/10)
Q + phase + delay*	3.93	4.73	3.38 ± 0.40	4.09 ± 0.41	15.8	−8.4% (9/10)

not explained by missing information alone: even LQF without a minimum green ($g_{min} = 1$), which uses nothing that Q + phase + delay does not observe, reaches $W_{all} = 3.98$ on the first test stream, against 4.71 for Q + phase + delay. The remaining gap is therefore at least partly a matter of learning and of the objective: episodes contain a fixed number of decisions rather than a fixed duration, so the discounted per-decision objective does not exactly match delay per unit time, and the agent learns to hold phases for longer than LQF does.

6.8 Sensitivity to the arrival model

Under the released arrival model, a phase change re-applies the step's arrival draw in each of its four ticks, so arrivals come in bursts exactly while the intersection is blocked. Real arrivals do not depend on the signal, and the DQN was trained with independent draws in every tick. Table 5 repeats the main comparison under that

model, with baselines re-tuned and tabular agents retrained under it. All controllers have lower delay, by 12% (LQF) to 30% (the transferred Q-table); the DQN's falls by about a quarter. Retraining matters little for the tabular agents: those trained for 3,000 episodes under the released model reach 4.33 under per-tick arrivals, against 4.38 when retrained. Under this model the DQN has the lowest W_{dec} of all controllers, 6.8% below LQF. Under W_{all} it is 14.5% better than Q (3000 ep.) (better than all 10 seeds; $p = 1.6 \times 10^{-7}$ over seeds), 8.4% better than Q + phase + delay (9 of 10 seeds; $p = 0.03$) and 19.0% better than fixed-time control. These comparisons pit a single DQN run against ten tabular seeds and therefore ignore the DQN's own seed-to-seed variability. LQF still has the lowest W_{all}; the DQN's is 16.7% higher. The design change of Section 6.7 again cuts switching by 71%, but its 6.6% reduction in W_{all} is not significant across seeds under this model (Welch $p = 0.06$). A smaller demand sweep (Appendix D) confirms that the learned agents degrade above the training demand. The arrival model thus decides whether the DQN beats the retrained tabular agents under full accounting, but not whether LQF beats all learned agents.

7 Discussion

Pixels or queues? In this environment the answer is nuanced. A convolutional network trained on raw rasters learned, without any notion of queues or phases, to read stop-line occupancy and the signal state and to act on them. On the measure it was developed for, it beats fixed-time control and tabular agents trained on the same environment, and when arrivals are generated independently in every tick it also has less clearance-inclusive delay than the retrained tabular agents (a single DQN run against ten tabular seeds). Yet a rule that uses only the two queue counts, the current phase and a minimum-green timer outperforms every learned agent on clearance-inclusive delay under both arrival models. This suggests that, at a single-lane isolated intersection, the additional detail in the raster offers little beyond what these few variables provide, in line with earlier findings that low-resolution inputs can suffice [12, 50]. Richer inputs are more likely to pay off where counts are insufficient: multiple lanes and turning movements, partial observability, heterogeneous vehicles, or coordination between intersections. We also note that our comparison is between two artifacts that differ in more than their input: reward, training budget, episode length and training arrival model all differ, and the DQN is a single training run.

Lessons for evaluating signal-control RL. Our results suggest a short checklist. (i) *Count every tick*: metrics and rewards that skip clearance intervals hide roughly a third to two thirds of the halted vehicle-ticks here and reward excessive switching. (ii) *Check how randomness is implemented*: reusing one arrival draw across the ticks of a phase change shifted the DQN's relative delay by about 14 percentage points against Q (3000 ep.) and about 16 against Q + phase + delay. (iii) *Include queue-based heuristics* such as LQF or max-pressure [38] and a properly timed fixed-time plan [41], tuned on separate data. (iv) *Use held-out arrival streams and common random numbers*, report confidence intervals, and compare at the level of training seeds. (v) *Report delay distributions*, not only means, since learned policies can trade average delay for long waits. (vi) *Stress-test demand*, because policies trained at one level can fail just above it. (vii) *Align the reward with the metric and give the agent the phase*, so that the cost of switching is both observable and penalized.

Limitations. The environment is deliberately simple: single-lane approaches without turns, unit speed, no yellow interval, no pedestrians, and a cap that blocks arrivals at full approaches and lets vehicles share the entry cell. We retained the released dynamics to stay faithful to the original implementation; their quantitative effects should not be transferred to real intersections without validation in microscopic simulators such as SUMO [24] or CityFlow [48]. We evaluated the released DQN checkpoint but did not retrain it: one update of the 256-filter network takes about 65 ms on our CPU, so reproducing the checkpoint's 5×10^5 updates would take roughly nine hours per seed and configuration, more than a day for even three seeds, which exceeded the

budget of this study. Consequently, we cannot say how much of the tabular improvements would carry over to the pixel-based agent, nor how much the DQN varies across training seeds. The stored DQN results could not be reproduced exactly, and the training schedule of the checkpoint is not recorded. Finally, all statements concern a single intersection.

Future work. Natural next steps are to retrain the DQN with a phase-aware, clearance-inclusive objective under several seeds and with modern improvements such as double and dueling estimators or prioritized replay [31, 37, 39]; to train a network on queue features under identical settings, isolating the effect of the input representation; to formulate the problem as a semi-Markov decision process with time-based discounting or average delay per unit time; and to repeat the pixels-versus-queues comparison with multi-lane approaches, turning movements and networks of intersections.

8 Conclusion

We compared a tabular Q-learning agent on queue counts with a deep Q-network on raw pixels for controlling an isolated intersection, and re-evaluated both far beyond the original protocol. The pixel-based agent learns to read stop-line occupancy and signal heads and, on the measure it was developed for, outperforms fixed-time control and the tabular learner. A fuller accounting that includes the clearance intervals, strong heuristic baselines, demand sweeps, delay tails and a second arrival model tells a more cautious story: the learned agents switch at costly moments, have more clearance-inclusive delay than a longest-queue rule in every condition we tested, degrade above their training demand, and their relative ranking depends on how arrivals are simulated. Giving the tabular agent the current phase and a clearance-inclusive reward sharply reduces its switching and closes part of the gap. We hope that the released environment, agents and evaluation scripts make it easy to test these lessons on stronger agents and richer traffic.

Code and data availability

The environment, training and evaluation scripts, trained agents and arrival streams are available at `https://github.com/zahraKrm/Deep-RL-Traffic-Signal-Control` [18]. The analysis scripts that produce every table and figure of this paper are provided as supplementary material.

Acknowledgements

Parts of the analysis code and of the manuscript text were prepared with the assistance of an AI system; the authors reviewed all results and text and take full responsibility for the content.

References

[1] Baher Abdulhai, Rob Pringle, and Grigoris J. Karakoulas. Reinforcement learning for true adaptive traffic signal control. *Journal of Transportation Engineering*, 129(3):278–285, 2003. doi: 10.1061/(ASCE)0733-947X(2003)129:3(278).

[2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[3] James Ault and Guni Sharon. Reinforcement learning benchmarks for traffic signal control. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[4] James Ault, Josiah P. Hanna, and Guni Sharon. Learning an interpretable traffic signal control policy. In *Proceedings of the 19th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 88–96, 2020.

[5] Ofer Biham, A. Alan Middleton, and Dov Levine. Self-organization and a dynamical transition in traffic-flow models. *Physical Review A*, 46(10):R6124–R6127, 1992. doi: 10.1103/PhysRevA.46.R6124.

[6] Alvaro Cabrejas-Egea, Shaun Howell, Maksis Knutins, and Colm Connaughton. Assessment of reward functions for reinforcement learning traffic signal control under real-world limitations. In *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2020. doi: 10.1109/SMC42975.2020.9283498.

[7] Chacha Chen, Hua Wei, Nan Xu, Guanjie Zheng, Ming Yang, Yuanhao Xiong, Kai Xu, and Zhenhui Li. Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3414–3421, 2020. doi: 10.1609/aaai.v34i04.5744.

[8] Tianshu Chu, Jie Wang, Lara Codecà, and Zhaojian Li. Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 21(3): 1086–1095, 2020.

[9] Samah El-Tantawy, Baher Abdulhai, and Hossam Abdelgawad. Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (MARLIN-ATSC): Methodology and large-scale application on downtown Toronto. *IEEE Transactions on Intelligent Transportation Systems*, 14(3): 1140–1150, 2013.

[10] Juntao Gao, Yulong Shen, Jia Liu, Minoru Ito, and Norio Shiratori. Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network. arXiv:1705.02755, 2017.

[11] Wade Genders and Saiedeh Razavi. Using a deep reinforcement learning agent for traffic signal control. arXiv:1611.01142, 2016.

[12] Wade Genders and Saiedeh Razavi. Evaluating reinforcement learning state representations for adaptive traffic signal control. *Procedia Computer Science*, 130:26–33, 2018. doi: 10.1016/j.procs.2018.04.008.

[13] Carlos Gershenson. Self-organizing traffic lights. *Complex Systems*, 16(1):29–53, 2005.

[14] Samuel Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and understanding Atari agents. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1792–1801, 2018.

[15] Ammar Haydari and Yasin Yılmaz. Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(1):11–32, 2022.

[16] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[17] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and M. C. Royle. The SCOOT on-line traffic signal optimisation technique. *Traffic Engineering & Control*, 23(4):190–192, 1982.

[18] Zahra Karami. Deep-RL-Traffic-Signal-Control: Deep reinforcement learning for intelligent traffic signal control using DQN and Q-learning. GitHub repository, 2026. URL https://github.com/zahraKrm/Deep-RL-Traffic-Signal-Control.

[19] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[20] Peter Koonce, Lee Rodegerdts, et al. Traffic signal timing manual. Technical Report FHWA-HOP-08-024, Federal Highway Administration, Washington, DC, 2008.

[21] Li Li, Yisheng Lv, and Fei-Yue Wang. Traffic signal timing via deep reinforcement learning. *IEEE/CAA Journal of Automatica Sinica*, 3(3):247–254, 2016.

[22] Xiaoyuan Liang, Xunsheng Du, Guiling Wang, and Zhu Han. A deep reinforcement learning network for traffic light cycle control. *IEEE Transactions on Vehicular Technology*, 68(2):1243–1253, 2019. doi: 10.1109/TVT.2018.2890726.

[23] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.

[24] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using SUMO. In *21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582, 2018. doi: 10.1109/ITSC.2018.8569938.

[25] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[26] Seyed Sajad Mousavi, Michael Schukat, and Enda Howley. Traffic light control using deep policy-gradient and value-function-based reinforcement learning. *IET Intelligent Transport Systems*, 11(7):417–423, 2017.

[27] Kai Nagel and Michael Schreckenberg. A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12):2221–2229, 1992. doi: 10.1051/jp1:1992277.

[28] Mohammad Noaeen, Atharva Naik, Liana Goodman, Jared Crebo, Taimoor Abrar, Behrouz Far, Zahra Shakeri Hossein Abad, and Ana L. C. Bazzan. Reinforcement learning in urban network traffic signal control: A systematic literature review. *Expert Systems with Applications*, 199:116830, 2022. doi: 10.1016/j.eswa.2022.116830.

[29] L. A. Prashanth and Shalabh Bhatnagar. Reinforcement learning with function approximation for traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 12(2):412–421, 2011. doi: 10.1109/TITS.2010.2091408.

[30] Faizan Rasheed, Kok-Lim Alvin Yau, Rafidah Md. Noor, Celimuge Wu, and Yeh-Ching Low. Deep reinforcement learning for traffic signal control: A review. *IEEE Access*, 8:208016–208044, 2020. doi: 10.1109/ACCESS.2020.3034141.

[31] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[32] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *International Conference on Learning Representations (ICLR), Workshop Track*, 2014.

[33] Arthur G. Sims and Kenneth W. Dobinson. The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits. *IEEE Transactions on Vehicular Technology*, 29(2):130–137, 1980. doi: 10.1109/T-VT.1980.23833.

[34] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

[35] U.S. Department of Energy, Vehicle Technologies Office. FOTW #1358: The average commuter experienced 54 hours of delay due to traffic congestion in 2022. Fact of the Week, 2 September 2024. Based on the Texas A&M Transportation Institute 2023 Urban Mobility Report, 2024. URL https://www.energy.gov/cmei/vehicles/articles/fotw-1358-sept-2-2024-average-commuter-experienced-54-hours-delay-due.

[36] Elise van der Pol and Frans A. Oliehoek. Coordinated deep reinforcement learners for traffic light control. In *NIPS 2016 Workshop on Learning, Inference and Control of Multi-Agent Systems*, 2016.

[37] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

[38] Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013. doi: 10.1016/j.trc.2013.08.014.

[39] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, 2016.

[40] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi: 10.1007/BF00992698.

[41] F. V. Webster. Traffic signal settings. Road Research Technical Paper 39, Road Research Laboratory, Her Majesty's Stationery Office, London, 1958.

[42] Hua Wei, Guanjie Zheng, Huaxiu Yao, and Zhenhui Li. IntelliLight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2496–2505, 2018. doi: 10.1145/3219819.3220096.

[43] Hua Wei, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu, and Zhenhui Li. PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1290–1298, 2019. doi: 10.1145/3292500.3330949.

[44] Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. A survey on traffic signal control methods. arXiv:1904.08117, 2019.

[45] Marco A. Wiering. Multi-agent reinforcement learning for traffic light control. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)*, pages 1151–1158, 2000.

[46] Kok-Lim Alvin Yau, Junaid Qadir, Hooi Ling Khoo, Mee Hong Ling, and Peter Komisarczuk. A survey on reinforcement learning models and algorithms for traffic signal control. *ACM Computing Surveys*, 50(3):34:1–34:38, 2017. doi: 10.1145/3068287.

[47] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European Conference on Computer Vision (ECCV)*, volume 8689 of *Lecture Notes in Computer Science*, pages 818–833, 2014.

[48] Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. CityFlow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW), Demo Track*, 2019. doi: 10.1145/3308558.3314139.

[49] Guanjie Zheng, Yuanhao Xiong, Xinshi Zang, Jie Feng, Hua Wei, Huichu Zhang, Yong Li, Kai Xu, and Zhenhui Li. Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1963–1972, 2019. doi: 10.1145/3357384.3357900.

[50] Guanjie Zheng, Xinshi Zang, Nan Xu, Hua Wei, Zhengyao Yu, Vikash Gayah, Kai Xu, and Zhenhui Li. Diagnosing reinforcement learning for traffic signal control. arXiv:1905.04716, 2019.

A Simulator details

Algorithm 1. One decision step of the cellular intersection (released arrival model)

Input: chosen phase a_t, previous phase a_{t-1}, arrival draws $u_{i,t}$ for the four approaches.

1: **if** $a_t \neq a_{t-1}$: set both axes to red and run TICK three times (clearance).
2: Set the axis of a_t to green and the other to red; run TICK once (decision tick).
3: Compute h_t, queues q_i and the reward r_t^{dev} (Eq. (1)) from the decision tick.

TICK: for each approach i in the order north, south, east, west:

a: **if** $u_{i,t} > 1-p$ and $q_i \leq S/2-2$: append a vehicle at the entry cell of i (otherwise, if $u_{i,t} > 1-p$, count a blocked arrival).
b: For each vehicle of i in order of arrival: count it towards the new queue q_i if it is upstream of the signal line; it is *halted* if the next cell holds a vehicle of i, or if its signal is red and the next cell is the signal line; otherwise it advances one cell. Vehicles reaching the far boundary leave.

In the per-tick arrival model of Section 6.8, step (a) uses a fresh draw in every tick.

The geometry follows the released code. For a grid of size S, the northbound lane is column $S/2+1$ (entering at row $S-1$), the southbound lane column $S/2-1$ (entering at row 1), the eastbound lane row $S/2+1$ (entering at column 1) and the westbound lane row $S/2-1$ (entering at column $S-1$). Signal heads sit at $(S/2\pm 2, S/2\pm 2)$ (coordinates rounded down), so each approach holds seven queueing cells upstream of its signal line on 20×20 and five on 15×15. The penalty threshold of r^{dev} is $S/4$ (5 and 3.75 vehicles, respectively). Three implementation details of the released code are retained to preserve its results exactly: arrival draws are indexed by decision step, so the same draw applies to each tick of a switching step; a vehicle inserted into a full approach shares the entry cell with the vehicle already there (Section 3.2); and removing a vehicle at the boundary skips the update of the following vehicle of the same approach in that tick, which also omits that vehicle from the queue count of that tick.

B Hyperparameters

Table 6: Hyperparameters of the agents as set in the released scripts, and of our retraining.

	Tabular Q-learning	Deep Q-network
Grid	15×15 (released), 20×20 (retrained)	20×20
State	$(q_{\mathrm{NS}}, q_{\mathrm{EW}})$ [+ current phase in ablation]	raster 20×20×3, scaled to $[0,1]$
Training reward	r^{dev} [r^{delay} in ablation]	queue-decrease reward (Section 3)
Training arrivals	draw per decision step (per tick in Section 6.8)	independent draw per tick
Discount γ	0.95	0.95
Learning rate	$\alpha = 0.1$	Adam, 10^{-3}, MSE loss
Exploration	ε: 1.0, ×0.995 per episode (floor 0.01 when retraining)	ε: 0.9, ×0.9998 per episode (released script)
Episodes × steps	100 × 200 (released script); 3000 × 200 (retrained)	1000 steps per episode; ≈ 500 episodes (from optimizer state)
Replay / batch	—	50,000 / 64, warm-up 1,000
Target network	—	synchronized every 3 episodes
Network	—	2×[conv 3×3, 256, ReLU, max-pool 2, dropout 0.2], FC 64, FC 2
Parameters	up to 40 × 40 × 2 table entries	744,898

C Results on the 15×15 grid

Table 7: Results on the native 15×15 **grid of the released Q-table** (5,000 held-out episodes; mean ± 95% CI half-width for single controllers, mean ± s.d. across ten seeds for retrained agents).

Controller	W_{dec}	W_{all}	d_{95}	Throughput	Switches	Blocked (%)
Fixed-time (Webster, g=6)	5.00 ± 0.03	5.72 ± 0.02	11.7	0.367	33.0	2.2
SOTL	3.46 ± 0.02	4.82 ± 0.02	13.1	0.367	26.7	1.8
LQF	2.32 ± 0.02	3.54 ± 0.02	10.9	0.371	31.1	1.5
Random	11.18 ± 0.06	14.33 ± 0.05	36.2	0.334	99.8	11.5
Q (released)	3.12 ± 0.03	5.14 ± 0.05	17.0	0.365	47.3	3.2
Q (100 ep.), 10 seeds	3.91 ± 0.36	6.34 ± 0.51	20.5	0.360	51.2	4.3
Q (3000 ep.), 10 seeds	3.20 ± 0.35	5.11 ± 0.42	16.6	0.366	40.7	2.8

Table 7 repeats the main comparison on the grid on which the released tabular agent was trained. The conclusions are the same as on 20×20: the released Q-table ($W_{\mathrm{dec}} = 3.12$, $W_{\mathrm{all}} = 5.14$) improves on fixed-time control under both measures but trails LQF, with a W_{all} 45.2% higher than LQF's 3.54, and tabular agents retrained for 3,000 episodes match it ($W_{\mathrm{all}} = 5.11$), whereas the released 100-episode budget does not ($W_{\mathrm{all}} = 6.34$). Note that 7 rather than 6 would be the best fixed-time green on this grid if tuned on W_{dec}.

D Demand sweep with independent per-tick arrivals

Under the per-tick arrival model (Section 6.8), with 250 episodes per demand level, the DQN's W_{all} at $p = 0.14$ is 6.09 against 5.42 for fixed-time control, 5.40 for the re-timed Webster plan and 4.42 for LQF; at $p = 0.20$ it is 14.78 against 7.10 for the re-timed plan. At $p = 0.02$ the DQN reaches 1.57, against 3.95 for

fixed-time control, 1.17 for Q (3000 ep.) retrained under this model and 0.64 for LQF. The qualitative picture of Figure 10 is therefore unchanged: the learned agents beat fixed-time control at and below the training demand but trail LQF, and they degrade rapidly above it.